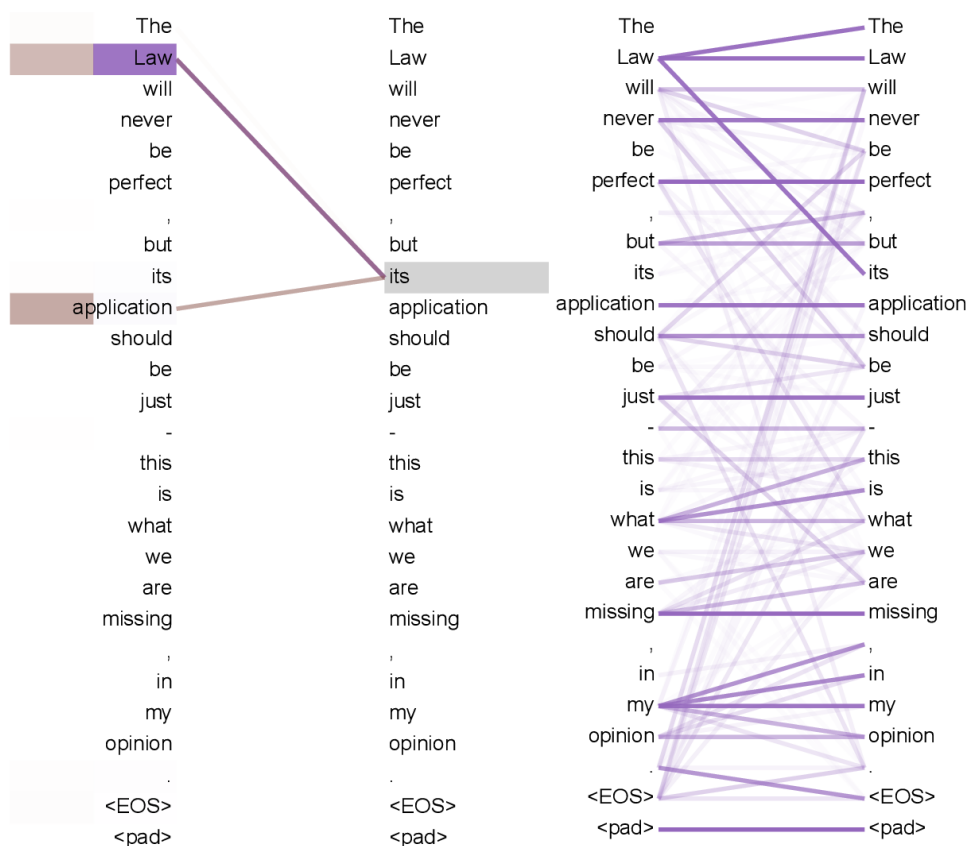


Mini-Transformer: um “ChatGPT simplificado” em RISC-V

Este documento descreve os projetos da disciplina de Introdução à Arquitetura de Computadores 2025/2026 (LEIC-A e LEIC-T).



Fonte: Vaswani et al., Attention Is All You Need, NIPS 2017

Histórico de versões

- v1.0: Enunciado publicado (20.04.2026).
- v1.1: Alterações ao Projeto 1
- v1.2: Alterações ao Projeto 2
- v1.3: Pequenos ajustes aos comentários no código exemplo no Projeto 2 (categorização *in/out* de argumentos a funções)
- v1.4: Incorporação da descrição detalhada do Projeto 3.

Introdução

Nos últimos anos, surgiram aplicações capazes de efetuar tarefas complexas relacionadas com texto – por exemplo, responder a perguntas, traduzir frases ou gerar texto automaticamente. Ferramentas como o ChatGPT¹ são exemplos desse tipo de sistemas. Muitos destes sistemas baseiam-se numa arquitetura de Inteligência Artificial baseada em redes neurais² chamada **Transformer**³.

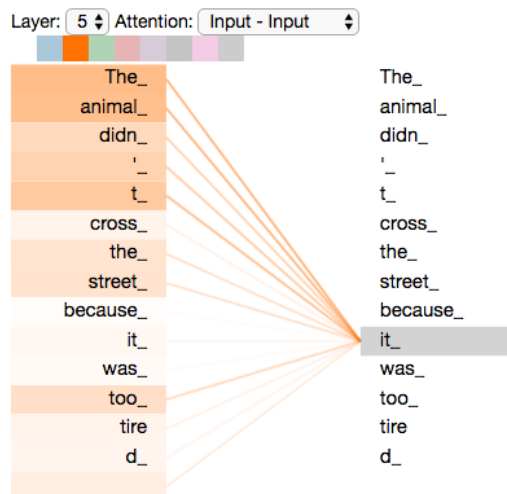
A ideia central destes modelos é o mecanismo de **atenção (attention)**: *para cada elemento da entrada (por exemplo, para cada palavra numa frase), o mecanismo decide quais são os outros elementos (palavras) mais relevantes*. Isto permite interpretar corretamente o significado das palavras tendo em conta o seu **contexto**, algo essencial em linguagem natural.

Vamos motivar este mecanismo de atenção com um exemplo⁴. Considera a seguinte frase:

“The animal didn’t cross the street because it was too tired.”

Uma questão importante é: a que se refere a palavra “it”? Será a “animal” ou a “street”? Para responder corretamente, é necessário perceber o contexto da frase. A palavra “tired” está mais associada a “animal” do que a “street”, pelo que “it” se refere ao animal. Isto mostra que, ao interpretar uma palavra, precisamos de olhar para outras palavras da frase, e que nem todas são igualmente importantes.

Um modelo baseado em atenção atribui diferentes níveis de importância (ou **pesos**) às palavras. Neste exemplo, a palavra “animal” terá uma ligação mais forte com “it”, como ilustrado na figura seguinte:



Para um computador fazer este tipo de raciocínio, cada palavra é representada por um **vetor de valores numéricos**, e os pesos de atenção são calculados através de operações matemáticas sobre esses vetores, em particular **produtos internos**. Em modelos reais, estes vetores são aprendidos automaticamente a partir

¹ <https://chatgpt.com>

² <https://developers.google.com/machine-learning/crash-course/neural-networks>

³ Vaswani *et al*, Attention Is All You Need, *NIPS 2017*

⁴ <https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/>

de grandes quantidades de dados. Existem vários detalhes adicionais que tornam estes modelos muito eficazes, mas, para já, a ideia essencial é a seguinte: *o mecanismo de atenção permite perceber o contexto de uma palavra numa frase através do cálculo de pesos de relevância entre as palavras, usando operações relativamente simples sobre vetores.*

Neste projeto, iremos focar-nos numa tarefa específica destes modelos: **a previsão do próximo token (palavra)**. Dada uma sequência de palavras, o objetivo do modelo é prever qual será a palavra seguinte. Esta é a tarefa fundamental por trás de muitos sistemas de geração de texto, incluindo chatbots como o ChatGPT.

Mini-Transformer

Neste projeto, irão implementar um modelo de Inteligência Artificial muito simples, inspirado no mecanismo de atenção apresentado anteriormente, a que chamaremos **Mini-Transformer**. O objetivo não é reproduzir um Transformer completo, mas sim construir um pequeno sistema que aplica, na prática, a ideia central da atenção: *comparar elementos de uma frase entre si, calcular a sua relevância, e usar essa informação para produzir um resultado.*

1. Entrada do modelo: frase → tokens → vetores

O modelo recebe como entrada uma **frase**. Para poder ser processada por um computador, a frase é dividida em unidades chamadas **tokens**. Neste projeto, vamos assumir que cada token corresponde a uma **palavra**.

O modelo utiliza um vocabulário fixo, isto é, um conjunto limitado de palavras (tokens) que conhece. A cada palavra do vocabulário é associada a um índice inteiro.

Por exemplo, a frase “the animal didn’t cross the street” pode ser representada como uma sequência de tokens:

["the", "animal", "didn't", "cross", "the", "street"]

Usando o vocabulário fornecido, cada token é convertido no índice correspondente, obtendo-se uma sequência de inteiros. Por exemplo:

[2, 3, 4, 5, 2, 6]

No entanto, estes inteiros servem apenas para identificar palavras e não contêm informação sobre o seu significado. Para que o modelo consiga comparar palavras e perceber relações entre elas, cada índice é convertido num vetor de números (designado por *embedding*). Por exemplo:

“the” → 2 → [0.1, 0.3, 0.7]

“animal” → 3 → [0.8, 0.2, 0.1]

...

Estes vetores permitem ao modelo calcular semelhanças entre palavras (por exemplo, através de produtos internos), o que é essencial para o funcionamento do mecanismo de atenção.

Assim, a frase passa a ser representada por uma **sequência de vetores**, que pode também ser vista como uma **matriz**. Chamando E a essa representação, temos:

$$E = [[0.1, 0.3, 0.7], [0.8, 0.2, 0.1], \dots]$$

Cada elemento (linha) de E corresponde ao vetor (*embedding*) de um token da frase.

Notem que os *embeddings* não são definidos ao acaso. Em modelos reais, são aprendidos automaticamente a partir de dados, ajustando os vetores de forma a que palavras que aparecem em contextos semelhantes tenham representações semelhantes. Neste projeto, no entanto, esses vetores serão fornecidos.

2. Como funciona o Mini-Transformer?

Recordemos que a sequência de vetores de entrada pode ser representada por uma **matriz** E , onde:

- cada elemento (linha) de E é o vetor (*embedding*) de um token (palavra)
- se a frase tiver n tokens e embeddings de dimensão d , então E tem dimensão $n \times d$

Assim, E **representa a frase inteira**.

A partir desta matriz E , o Mini-Transformer executa os seguintes passos:

Passo 1. Gerar representações intermédias (Q , K , V)

Para cada token i (isto é, para cada vetor i em E), o modelo calcula três novos vetores:

- Q_i (Query)
- K_i (Key)
- V_i (Value)

Assim, Q , K e V são matrizes com n linhas (uma por token), onde a linha i corresponde aos vetores Q_i , K_i e V_i , respetivamente.

Estas representações são obtidas através de transformações lineares simples:

$$Q = E \times W_Q$$

$$K = E \times W_K$$

$$V = E \times W_V$$

onde:

- E é a matriz de vetores de entrada

- W_Q , W_K , W_V são matrizes de pesos (fornecidas no projeto)

Interpretação. As matrizes W_Q , W_K e W_V transformam os embeddings originais em três representações diferentes. Em particular:

- As matrizes Q e K são usadas para calcular relevâncias entre palavras
- A matriz V representa a informação que será utilizada para construir a saída

Ou seja, cada palavra passa a ter três versões diferentes:

- uma para comparar com as outras (Q)
- uma para ser comparada (K)
- uma que contém a informação a usar na saída (V)

Passo 2. Calcular relevâncias.

Depois de calcular Q e K , o modelo mede a relevância entre palavras. Para isso, cada vetor de Q é comparado com todos os vetores K , usando o **produto interno**. Se quisermos analisar a palavra na posição i , então calculamos:

$$\text{score}(i, j) = Q_i \cdot K_j$$

para todos os valores de j .

Isto significa que escolhemos uma palavra da frase, pegamos no seu vetor Q_i , comparamos esse vetor com os vetores K_j de todas as palavras da frase e obtemos uma lista de valores, um por palavra.

Quanto maior for o valor obtido, maior é a relevância dessa palavra para a palavra que estamos a analisar.

Passo 3. Selecionar o mais relevante.

Depois de calcular todos os valores de relevância para uma palavra, o modelo seleciona o maior deles. Isto equivale a escolher a palavra da frase que é considerada mais relevante para a palavra que estamos a analisar. Neste projeto, essa escolha é feita usando uma função de máximo (**argmax**).

Passo 4. Produzir a saída.

Depois de escolher a palavra mais relevante, o modelo vai buscar o vetor V_i correspondente a essa palavra. Esse vetor V_i passa então a ser a nova representação da palavra que estamos a analisar.

Ou seja, Q e K servem para decidir *qual* a palavra mais relevante e V fornece a informação que será efetivamente usada.

Passo 5. Decisão final (classificação).

Neste projeto, para produzir a saída final, usamos o vetor correspondente à última palavra da frase. A partir desse vetor, o modelo escolhe uma palavra do vocabulário *como previsão para o próximo token* (isto é, a próxima palavra da frase). Essa escolha é feita comparando esse vetor com os vetores das palavras do vocabulário e selecionando a palavra mais semelhante.

Nota: Embora usemos apenas o vetor correspondente à última palavra para produzir a saída final, esse vetor já incorpora informação das restantes palavras da frase. Isto acontece porque, nos passos anteriores, cada palavra foi comparada com todas as outras através do mecanismo de atenção.

3. Exemplo numérico

Para ilustrar o funcionamento do modelo, vamos considerar a frase original, mas simplificada, considerando apenas os três primeiros tokens:

["the", "animal", "didn't"]

O objetivo é descobrir qual a palavra seguinte (tarefa de previsão do próximo token), que está na base de muitos sistemas de geração de texto.

Passo 1. Representação de entrada

Suponhamos que os embeddings são:

$$\begin{aligned} E = [\\ & [1,0], \leftarrow \text{"the"} \\ & [0,1], \leftarrow \text{"animal"} \\ & [1,1] \leftarrow \text{"didn't"} \\] \end{aligned}$$

Considere as seguintes matrizes de pesos:

$$\begin{aligned} W_Q = [\\ & [1,0], \\ & [0,1] \\] \end{aligned}$$

$$\begin{aligned} W_K = [\\ & [-2,0], \\ & [1,1] \end{aligned}$$

]

$W_V = [$

[0,1],

[2,0]

]

Passo 2. Cálculo de Q , K e V

$Q = [$

[1,0], $\leftarrow$ "the"

[0,1], $\leftarrow$ "animal"

[1,1] $\leftarrow$ "didn't"

]

$K = [$

[-2,0], $\leftarrow$ "the"

[1,1], $\leftarrow$ "animal"

[-1,1] $\leftarrow$ "didn't"

]

$V = [$

[0,1],

[2,0],

[2,1]

]

Passo 3. Calcular relevância (para a última palavra)

A última palavra é “didn't”, logo:

$Q_3 = [1,1]$

Calculamos os produtos internos:

$$Q_3 \cdot K_1 = [1,1] \cdot [-2,0] = -2 \leftarrow \text{"the"}$$

$$Q_3 \cdot K_2 = [1,1] \cdot [1,1] = 2 \leftarrow \text{"animal"}$$

$$Q_3 \cdot K_3 = [1,1] \cdot [-1,1] = 0 \leftarrow \text{"didn't"}$$

Passo 4. Selecionar o mais relevante

O maior valor é:

$2 \rightarrow \text{"animal"}$

Passo 5. Produzir a saída

Selecionamos:

$$V_2 = [2,0]$$

Passo 6. Decisão final

Suponhamos que temos no vocabulário:

$$\text{"the"} \rightarrow [1,0]$$

$$\text{"animal"} \rightarrow [0,1]$$

$$\text{"didn't"} \rightarrow [1,1]$$

$$\text{"cross"} \rightarrow [2,0]$$

$$\text{"street"} \rightarrow [1,2]$$

Comparando o vetor final $[2,0]$ com os vetores do vocabulário, o mais semelhante é:

"cross"

Projeto 1: Primeiros módulos

Neste primeiro projeto, irão implementar um conjunto de operações fundamentais sobre vetores e memória, que serão utilizadas no Projeto 2 para construir o Mini-Transformer.

O objetivo é consolidar o domínio de:

- operações sobre vetores em Assembly RISC-V;
- manipulação de memória através de ponteiros;
- controlo de fluxo (condições e ciclos).

Cada exercício corresponde a uma operação essencial do modelo de atenção, como comparar vetores ou selecionar resultados. Cada exercício deve ser implementado como um programa autónomo, pelo que a entrega consistirá em vários ficheiros independentes, um por operação.

Outras indicações úteis:

- Devem utilizar a **extensão de multiplicação de RISC-V** (extensão M).
- Todos os inteiros utilizados no projeto são **inteiros com sinal** (*signed int*, em contraste com *unsigned int*).
- Para submissões de excelência: serão valorizadas as implementações que fazem deteção de *overflow*, isto é, cálculos que ultrapassam os limites dos inteiros.

Seleção de elemento (select)

Esta função, implementada na zona de memória com a etiqueta **select**, **seleciona o elemento de um vetor**. Esta função recebe um vetor de inteiros e um índice e devolve o valor do vetor nessa posição.

➤ Argumentos

- **a1** (int *): Ponteiro para o início do vetor de inteiros
- **a2** (int): Número de inteiros no vetor.
- **a3** (int): índice do elemento

➤ Valor de Retorno

- **a0** (int): Código de estado de retorno
 - Código 0: sucesso
 - Código 50: argumento inválido (tamanho < 1)
 - Código 100: índice fora dos limites
- **a1** (int): Valor do elemento

Índice do maior elemento (argmax)

Esta função, implementada na zona de memória com a etiqueta **argmax**, recebe um vetor de inteiros e retorna o **índice do maior elemento**. Se existirem múltiplos elementos com o mesmo valor máximo, deve ser devolvido o menor índice entre eles.

Por exemplo, para o vetor de inteiros $[-6, -1, 6, 1]$, o resultado deve ser 2 (índice de 6). Para o vetor $[6, 1, 6, 1]$, o resultado deve ser 0, pois o valor 6 aparece pela primeira vez nessa posição.

➤ **Argumentos**

- **a1** (int *): Ponteiro para o início do vetor de inteiros.
- **a2** (int): Número de inteiros no vetor.

➤ **Valor de Retorno**

- **a0** (int): Código de estado de retorno
 - Código 0: sucesso
 - Código 50: argumento inválido (tamanho < 1)
- **a1** (int): Índice do maior elemento do vetor (em caso de empate, o menor índice).

Dot Product (dot)

A função `dot`, implementada na zona de memória com etiqueta **dot**, calcula o produto interno entre dois vetores de inteiros.

Dados dois vetores:

$A = [a_1, a_2, \dots, a_n]$

$B = [b_1, b_2, \dots, b_n]$

O resultado é:

$\text{dot}(A, B) = a_1 \cdot b_1 + a_2 \cdot b_2 + \dots + a_n \cdot b_n$

➤ **Argumentos**

- **a1** (int *): Ponteiro para vetor A
- **a2** (int *): Ponteiro para vetor B
- **a3** (int): Número de elementos de cada vetor

➤ **Valores de Retorno**

- **a0** (int): Código de estado de retorno
 - Código 0: sucesso
 - Código 50: argumento inválido (tamanho < 1)
 - Código 200: *overflow* detectado (**pontos extra**)
- **a1** (int): Resultado do produto interno

Nota final: As funções implementadas neste projeto serão reutilizadas no Projeto 2, nomeadamente:

- **dot**, para cálculo de relevâncias ($Q \cdot K$),
- **argmax**, para seleção do elemento mais relevante,

- `select`, para obtenção do valor V correspondente.

Em conjunto, estas funções permitem implementar o núcleo do mecanismo de atenção.

Projeto 2: Compor a Rede Neuronal

Neste projeto, irão construir o **Mini-Transformer completo**. O objetivo é implementar a pipeline completa do mecanismo de atenção, permitindo ao modelo prever o próximo token (palavra) de uma frase.

IMPORTANTE: Devem seguir a especificação descrita neste enunciado *rigorosamente*, especialmente o *nome* das etiquetas das funções pedidas e que registos utilizam como argumentos de entrada e saída. A avaliação será feita maioritariamente por testes automáticos, que falham caso a especificação não seja seguida. Devem também respeitar as convenções de utilização de registos. Se tiverem alguma dúvida relativamente à especificação, não hesitem em falar com o vosso docente de laboratório.

Objetivo

Dada uma sequência de tokens representada por uma matriz de embeddings E , o modelo deverá:

1. Obter a representação matricial da frase de entrada;
2. Calcular as matrizes Q , K e V ;
3. Calcular as relevâncias entre palavras;
4. Selecionar a palavra mais relevante para a última posição;
5. Produzir o vetor de saída correspondente;
6. Prever o próximo token com base nesse vetor

Pipeline do modelo

O modelo executa os seguintes passos:

Frase $\rightarrow$ *tokens* $\rightarrow$ *matriz* $E \rightarrow Q, K, V \rightarrow$ *scores* $\rightarrow$ *seleção* $\rightarrow$ *vetor final* $\rightarrow$ *previsão*

Tarefas a implementar

1. Leitura de ficheiros

O programa deverá ler o conteúdo de 6 ficheiros, categorizados em dois tipos diferentes:

- **Ficheiros com palavras**
 - Em ficheiros com palavras, cada linha do ficheiro contém uma e apenas uma palavra (*token*). Todas as palavras terminam com um caractere de nova linha (*newline*, ou $\backslash n$), incluindo a última palavra.
 - Lista de ficheiros com palavras:
 - **vocab.txt**: Ficheiro com o vocabulário.
 - **input.txt**: Ficheiro com a frase de input a ser analisada.
- **Ficheiros com uma matriz**
 - Em ficheiros com matrizes, cada linha do ficheiro contém uma linha da matriz. Colunas são separadas por um único espaço em branco. Todas as linhas terminam com um caractere de nova linha, incluindo a última linha. As matrizes serão compostas por inteiros com sinal (isto é, tanto números positivos como negativos, sempre inteiros).

- Lista de ficheiros com matrizes:
 - **embeddings.txt**: Ficheiro com a matriz de embeddings do vocabulário. Cada linha corresponde ao vetor de embeddings da palavra correspondente no vocabulário (primeira linha corresponde ao embedding da primeira palavra no vocabulário, segunda linha ao embedding da segunda palavra, etc.).
 - **W_Q.txt**: Ficheiro com os pesos da matriz W_Q .
 - **W_K.txt**: Ficheiro com os pesos da matriz W_K .
 - **W_V.txt**: Ficheiro com os pesos da matriz W_V .

Exemplos de cada um dos formados:

Ficheiro com palavras:

```
the
robot
finds
```

Ficheiro com uma matriz:

```
-2 1 1 -2
-8 -6 -8 -1
0 -5 -1 1
11 0 -1 9
```

Podem assumir que:

- Todos os ficheiros dados ao programa serão bem formados, não sendo necessária lógica para lidar com erros (por exemplo ficheiro não encontrado ou com formato fora do esperado).
- O conteúdo de qualquer ficheiro não irá ultrapassar os 1024 caracteres.
- O vocabulário não terá mais que 100 palavras.
- O ficheiro de input não terá mais que 10 palavras.
- Todas as palavras encontradas no ficheiro de input encontram-se também no vocabulário.
- Todas as matrizes terão 4 colunas (isto é, *dimensão* 4).
- A matriz de embeddings terá tantas linhas quanto o número total de palavras (e portanto linhas) no vocabulário.
- Todos os ficheiros terminam com o caractere *End Of File*, ou *EOF*, cujo número ASCII é 0.

Esta tarefa implica implementar a função **read_file**:

```
# Read from a text file into a buffer.
# (in)      a0: filename address (char*)
# (in/out) a1: destination buffer
# (in)      a2: maximum number of bytes to read
```

read_file:

2. Construção das matrizes através do conteúdo dos ficheiros

Depois da leitura de ficheiros, deverá utilizar o conteúdo (strings) de cada um dos ficheiros com matrizes para construir as respetivas matrizes de inteiros. As matrizes devem ser armazenadas em memória como arrays lineares de inteiros.

Esta tarefa implica implementar a função **parse_matrix_buffer**:

```
# Assumes the matrix is stored in the buffer as space-separated
integers.
# Assumes columns are separated by 1 space (' '), and rows by 1
newline ('\n').
# Assumes only signed integers are provided.
# (in/out) a0: address of the matrix to fill (int*)
# (out)     a1: number of rows in the matrix (int)
# (in)     a1: address of the buffer containing the matrix data
(char*)
parse_matrix_buffer:
```

3. Construção do vetor de índices da frase

Antes da construção da matriz de embeddings da frase, terá de encontrar a linha de cada palavra da frase de input no vocabulário. As linhas encontradas correspondem aos *índices* das palavras, e serão depois utilizadas para encontrar os embeddings das palavras.

Esta tarefa implica implementar a função **tokens_to_indices**:

```
# Converts the input tokens into their corresponding indices in the
vocabulary.
# (in/out) a0: address of input indices vector to fill (int*)
# (out)     a1: size of input indices vector (number of tokens in
input)
# (in)     a2: address to input buffer
# (in)     a3: address to vocabulary buffer
tokens_to_indices:
```

4. Construção da matriz de embeddings da frase

Agora que tem a matriz de embeddings do vocabulário e o vetor de índices de input, deverá construir a matriz de embeddings E de input. Cada índice no vetor de índices corresponde a uma linha (vetor) da

matriz de embeddings do vocabulário. Assim, a matriz de embeddings de input deverá, no final, ter tantas linhas quantas o vetor de índices.

Esta tarefa implica implementar a função **build_input_embeddings_matrix**:

```
# (in/out) a0: address of the output matrix to fill (int*)
# (in)      a1: address of the vocabulary embeddings matrix (int*)
# (in)      a2: address of the input indices array (int*)
# (in)      a3: number of tokens in the input (int)
build_input_embeddings_matrix:
```

5. *Multiplificação de matrizes*

O objetivo desta tarefa é construir as matrizes Q , K , e V :

$$\begin{aligned} Q &= E \times W_Q \\ K &= E \times W_K \\ V &= E \times W_V \end{aligned}$$

Esta tarefa implica implementar a função de multiplicação matriz $\times$ matriz **matrix_multiply**:

```
# (in/out) a0: address of the output matrix to fill (int*)
# (in)      a1: address of the first matrix (int*)
# (in)      a2: #rows of the first matrix (int)
# (in)      a3: #columns of the first matrix (int)
# (in)      a4: address of the second matrix (int*)
# (in)      a5: #rows of the second matrix (int)
# (in)      a6: #columns of the second matrix (int)
matrix_multiply:
```

6. *Cálculo de relevâncias*

Para a última palavra da frase (posição $n - 1$), calcular:

$$score(j) = Q[n-1] \cdot K[j]$$

para todos os valores de j (todas as posições da frase), em que “.” representa o produto interno (dot product).

Nota: o código esqueleto facultado já contém uma implementação da função de produto interno na etiqueta **dot.**

Esta tarefa implica implementar a função **compute_scores**:

```
# (in/out) a0: address of the output scores vector to fill (int*)
# (in)      a1: address of Q matrix (int*)
# (in)      a2: address of K matrix (int*)
# (in)      a3: #rows of Q and K (int)
# (in)      a4: #columns of Q and K (int)
# (in)      a5: target token index for which we want to compute the
score (int)
compute_scores:
```

7. *Produção do vetor de saída*

Depois de calcular o vetor de relevância, deverá seleccionar o índice com a maior pontuação. **Para isso, deverá utilizar a função *argmax* já proporcionada no código esqueleto facultado.** Usando o índice vindo da função *argmax*, deverá escolher o vetor apropriado da matriz V .

Esta tarefa implica implementar a função *select_vector_in_matrix*:

```
# (out) a0: address of the selected vector (int*)
# (in)   a1: address of matrix (int*)
# (in)   a2: #rows (int)
# (in)   a3: #cols (int)
# (in)   a4: target row
select_vector_in_matrix:
```

8. *Decisão final*

Deverá agora comparar o vetor final obtido com todos os embeddings do vocabulário e escolher a palavra cujo embedding seja mais semelhante. A comparação deve ser feita com o produto interno: produtos internos maiores irão equivaler a vetores mais semelhantes. Isto é, se A e B tiverem um produto interno maior que A e C , então A é mais semelhante a B do que a C .

O vetor de embeddings no vocabulário mais semelhante ao vetor final corresponde à próxima palavra seleccionada pelo modelo, isto é, à previsão do próximo token.

Esta tarefa implica implementar a função *decide_next_token*:

```
# (out) a0: index of the predicted token in the vocabulary (int)
# (in)   a0: address of target vector (int*)
# (in)   a1: vocabulary embeddings address (int*)
# (in)   a2: number of tokens in vocabulary (int)
```

```
decide_next_token:
```

9. Impressão do resultado final na consola e terminação

A última tarefa corresponde à impressão da palavra seleccionada como próximo token na consola. Para isso, deverá utilizar a função **já disponibilizada no esqueleto da solução: `print_predicted_token`**. Finalmente, deverão chamar a chamada de sistema *exit* com o código 0.

Projeto 3: Processador de 16 bits com aceleração DOT

Neste projeto, irão construir um pequeno processador especializado em algumas das operações fundamentais do Mini-Transformer desenvolvido nos projetos anteriores. Este processador será uma versão simplificada de um acelerador de Inteligência Artificial, semelhante aos que são utilizados em hardware moderno para acelerar modelos baseados em atenção. Em particular, este sistema irá suportar operações essenciais como produtos internos entre vetores, que estão no núcleo do mecanismo de atenção.

Este processador pode ser visto como uma versão muito simplificada de uma unidade de processamento de tensores (Tensor Processing Unit, TPU). Ao contrário da implementação em software realizada em RISC-V nos projetos anteriores, neste caso as operações serão implementadas diretamente em hardware, permitindo uma execução mais eficiente. O objetivo é compreender como operações de alto nível usadas em modelos de Inteligência Artificial podem ser mapeadas para circuitos digitais.

1. Objetivos

O objetivo deste projeto é aprofundar o conhecimento sobre a construção de sistemas digitais e os princípios fundamentais do funcionamento de um processador.

Este enunciado assume que os alunos já resolveram o guião laboratorial sobre Logisim. A partir das soluções construídas nesse guião, os alunos deverão conceber e implementar um processador simples de 16 bits, capaz de executar programas armazenados numa ROM.

Este processador incluirá duas instruções especializadas — **dot** e **dota** — que permitem calcular o produto escalar entre vetores. Estas operações estão diretamente relacionadas com o mecanismo de atenção estudado no Projeto 2, sendo operações fundamentais em modelos modernos de Inteligência Artificial.

2. Requisitos

Para a entrega final do Projeto 3, deverão conceber e implementar no simulador Logisim um processador de 16 bits (isto é, onde o tamanho da palavra é 16 bits), capaz de executar programas contidos numa ROM. Cada palavra da ROM deve conter uma instrução com tamanho fixo de 16 bits. O processador deverá seguir um modelo de execução *single-cycle*, em que cada instrução é executada num único ciclo de relógio.

2.1 Registos

O processador deverá incluir um pequeno banco de registos:

- 8 registos (R0 a R7)
- cada registo com 16 bits

2.2 Conjunto de instruções (ISA)

O processador deverá suportar as seguintes instruções:

=====

li rd, imediato

Carrega um valor imediato para um registo, em que o imediato é um inteiro com sinal (em complemento para dois):

$R[rd] \leftarrow \text{imediato}$

=====

add rd, rs

Soma dois registos:

$R[rd] \leftarrow R[rd] + R[rs]$

=====

dot rd, rs

Calcula o produto escalar de dois vetores de dimensão 2:

$R[rd] \leftarrow (R[rd] \times R[rs]) + (R[rd+1] \times R[rs+1])$

Na notação anterior, $R[x]$ significa o registo com o número x . Como tal, a instrução **dot** utiliza pares consecutivos de registos como vetores. Por exemplo:

dot R0, R2

corresponde a:

$R0 \leftarrow R0 \times R2 + R1 \times R3$

O resultado deverá ser armazenado em **rd**, considerando apenas os 16 bits menos significativos (sem necessidade de assinalar valores de transporte ou excesso – *overflow*).

=====

dota rd, rs1, rs2

Calcula o produto escalar acumulado de dois vetores de dimensão 2:

$R[rd] \leftarrow R[rd] + (R[rs1] \times R[rs2]) + (R[rs1+1] \times R[rs2+1])$

Na notação anterior, $R[x]$ significa o registo com o número x . Como tal, a instrução **dota** utiliza pares consecutivos de registos como vetores. Por exemplo:

dota R0, R1, R3

corresponde a:

$R0 \leftarrow R0 + R1 \times R3 + R2 \times R4$

O resultado deverá ser armazenado em **rd**, considerando apenas os 16 bits menos significativos (sem necessidade de assinalar valores de transporte ou excesso – *overflow*).

2.3 Exemplo de programa

O programa seguinte calcula o produto escalar entre quatro vetores:

```
li  R0, 2
li  R1, 3
li  R2, 4
li  R3, 2
li  R4, 1
li  R5, 5
li  R6, 2
li  R7, 2
add R3, R1
dot R0, R2
dota R0, R4, R6
```

Resultado esperado:

$$R0 = 2 \times 4 + 3 \times 5 + 1 \times 2 + 5 \times 2 = 35.$$

2.4 Liberdade de implementação

Cada grupo tem liberdade para:

- definir o formato das instruções (codificação em linguagem máquina)
- escolher a organização interna do datapath
- definir sinais de controlo

Devem, no entanto, justificar as decisões tomadas, tendo em conta critérios como:

- simplicidade do circuito e clareza do desenho
- número de componentes
- expressividade da ISA
- extensibilidade da arquitetura

Os alunos devem incluir um ficheiro `README.txt` curto (1 página no máximo) onde definem o formato das instruções e justificam as principais opções tomadas à luz dos critérios atrás mencionados. Devem incluir também um exemplo de codificação em linguagem máquina para cada instrução.

3. Entrega

Os alunos devem entregar um ficheiro zip com dois ficheiros:

- o ficheiro `projeto3.circ` desenvolvido, e
- o ficheiro `README.txt`.

Considerações práticas

O projeto deve ser desenvolvido no simulador **Ripes** (1º e 2º projetos) e ferramenta **Logisim** (3º projeto), que serão apresentados nas aulas laboratoriais. O processador simulado em Ripes deve ser um **RISC-V de 32 bits**, de acordo com a arquitetura estudada nas aulas teóricas e laboratoriais. Pode ser usada a extensão M do RISC-V.

No caso dos 1º e 2º projetos, forneceremos os ficheiros de código iniciais (através do Fénix). As soluções devem **obrigatoriamente** ser construídas sobre esses ficheiros, respeitando a estrutura fornecida inicialmente.

Entregas e avaliação

Os projetos são submetidos no fénix, com os seguintes prazos:

- Projeto 1: 23h59 de 30/abril
- Projeto 2: 23h59 de 22/maio
- Projeto 3: 23h59 de 8/junho

O peso de cada projeto na nota final pode ser consultado nos métodos de avaliação no fenix. Nota mínima de 9,0.

Embora o projeto seja desenvolvido em grupos de 3 estudantes, haverá na semana de discussões um teste prático individual e discussão cujo objetivo é aferir se cada membro do grupo contribuiu e domina as soluções que foram submetidas pelo grupo. O teste individual inclui modificações ao projeto submetido pelo grupo e possivelmente o desenvolvimento de novas funcionalidades.

Usaremos software de deteção de cópias (abarcando uso de AI/ChatGPT). Casos de plágio seguem os procedimentos do regulamento de avaliação do IST.